SKILLSFORGE

SkillsForge Second Stage Selection

Track: Backend Development

Congratulations on successfully passing the initial stage of our selection process for the internship position at SkillsForge. We are impressed with your qualifications and would like to invite you to the second stage, which involves a practical task.

This task will assess your technical skills and problem-solving abilities.

Please note that your chosen tasks will not affect your chances of being selected for the position. We want to provide you with an opportunity to showcase your skills and approach to problem-solving, regardless of the specific project.

Answer all questions from section one and one question from section two

Section one

1) Given an integer K and an unsorted list of size N, write an algorithm that returns the position of the 2 numbers in the list that add up to number K.

The algorithm must use O(N) space and O(N) time.
Return the 0-indexed positions (indices) of the two numbers as a pair (e.g., (index1, index2)). Each number in the list can only be used once per pair.
You can assume that each data set will contain at least one solution (you will never have to return null / handle cases where no two numbers add up to K).

Test Cases:

```
assert two_sum_unsorted([2, 7, 11, 15], 9) == (0, 1)
assert two_sum_unsorted([3, 2, 4], 6) == (1, 2)
assert two_sum_unsorted([3, 3], 6) == (0, 1)
assert two_sum_unsorted([1, 5, 8, 3, 2], 11) == (2, 3)
```

2) Looking at the question above but assuming that the list is sorted in descending order, return the position of the 2 numbers in the list that add up to K.

The algorithm must use O(1) space and O(N) time.
The list is guaranteed to be sorted in descending order (e.g., [15, 11, 7, 2]). Auxiliary space must be strictly O(1) without using extra data structures such as hash tables or sets.
You can assume that each data set will contain at least one solution (you will never have to return null / handle cases where no two numbers add up to K).

Test Cases:

```
assert two_sum_descending([15, 11, 7, 2], 9) == (2, 3)
assert two_sum_descending([10, 8, 5, 3, 2, 1], 10) == (1, 4)
assert two_sum_descending([20, 14, 11, 8, 4, 1], 15) in [(1, 5), (2, 4)]
```

```
assert two_sum_descending([9, 7, 4, 3, 1], 10) in [(0, 4), (1, 3)]
```

Section Two

Project One: Personal Expense Tracker API

Develop an API where users can log personal expenses, categorise spending, and track financial summaries.

Features:

- CRUD operations for expenses (title, amount, category, date, payment method)
- CRUD operations for expense categories (e.g., Food, Transport, Utilities, Entertainment)
- Filter expenses by category, date range, or payment method
- Get spending summaries (total amount spent, total spent per category)
- Sort expenses by date or amount (ascending/descending)

Additional Features (Nice to Have):

- Monthly budget limit setting and an alert/flag when total spending exceeds the budget
- User authentication (each user only manages their own expenses)
- Pagination for expense listings
- Export expenses summary (CSV or JSON report)

Project Two: Event RSVP & Booking API

Develop an API where organizers can create events and attendees can RSVP or reserve a spot.

Features:

- CRUD operations for events (title, description, venue/online link, date & time, max capacity)
- CRUD operations for attendees (name, email, phone number)
- RSVP / registration endpoint (reserve a spot for an attendee)
- Capacity validation (prevent RSVP when event capacity is reached)
- View all attendees registered for a given event, or all events a user RSVP'd to

Additional Features (Nice to have):

- Cancel RSVP functionality (freeing up the spot)
- Waitlist support when an event is fully booked
- Filter events by upcoming status or date range
- User authentication (organizer vs. attendee roles)

SKILLSFORGE

Submission Guidelines:

- Write your program in any language of your choosing
- Work with any database of your choosing. In-memory data structures like Lists can also be used
- If there is any relevant codebase to your submission, push it to GitHub and provide the link in the email and report
- Push your well-formatted program, answers to (1) and (2) to a public GitHub Repo
- Include a README that explains how to run your program including details of your program logic
- A Postman / Swagger documentation of your API
- **Nice To Have:** For submissions that require specific environments to run, it would be best to dockerize the application.

Submission Deadline:

Please submit your completed task on or before October 31st 2026, using this submission form. Note that late submissions will not be considered.

We look forward to reviewing your work and discussing it further during the next stage of the selection process. If you have any questions or need clarification, please don't hesitate to reach out to us by replying to this email.